



Python基础



1

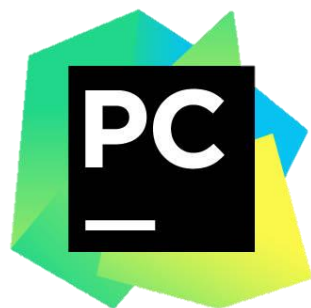
环境概述

1.1 环境概述 各类IDE: Python IDE



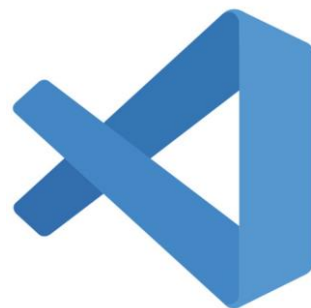
IDLE

是一个纯 Python 下使用 Tkinter 编写的相当基本的 IDE，具备基本的 IDE 的功能，是非商业 Python 开发的不错的选择。



Pycharm

是一种 Python IDE，带有一整套可以帮助用户在使用 Python 语言开发时提高其效率的工具。此外，该 IDE 提供了一些高级功能，以用于支持 Django 框架下的专业 Web 开发。



Visual Studio Code

开源 IDE 工具，它可以作为很多编程语言的 IDE，对于编程专业的学生很合适。具有丰富的扩展性。



Anaconda

Anaconda 适用于科学计算和数据分析，开源免费。



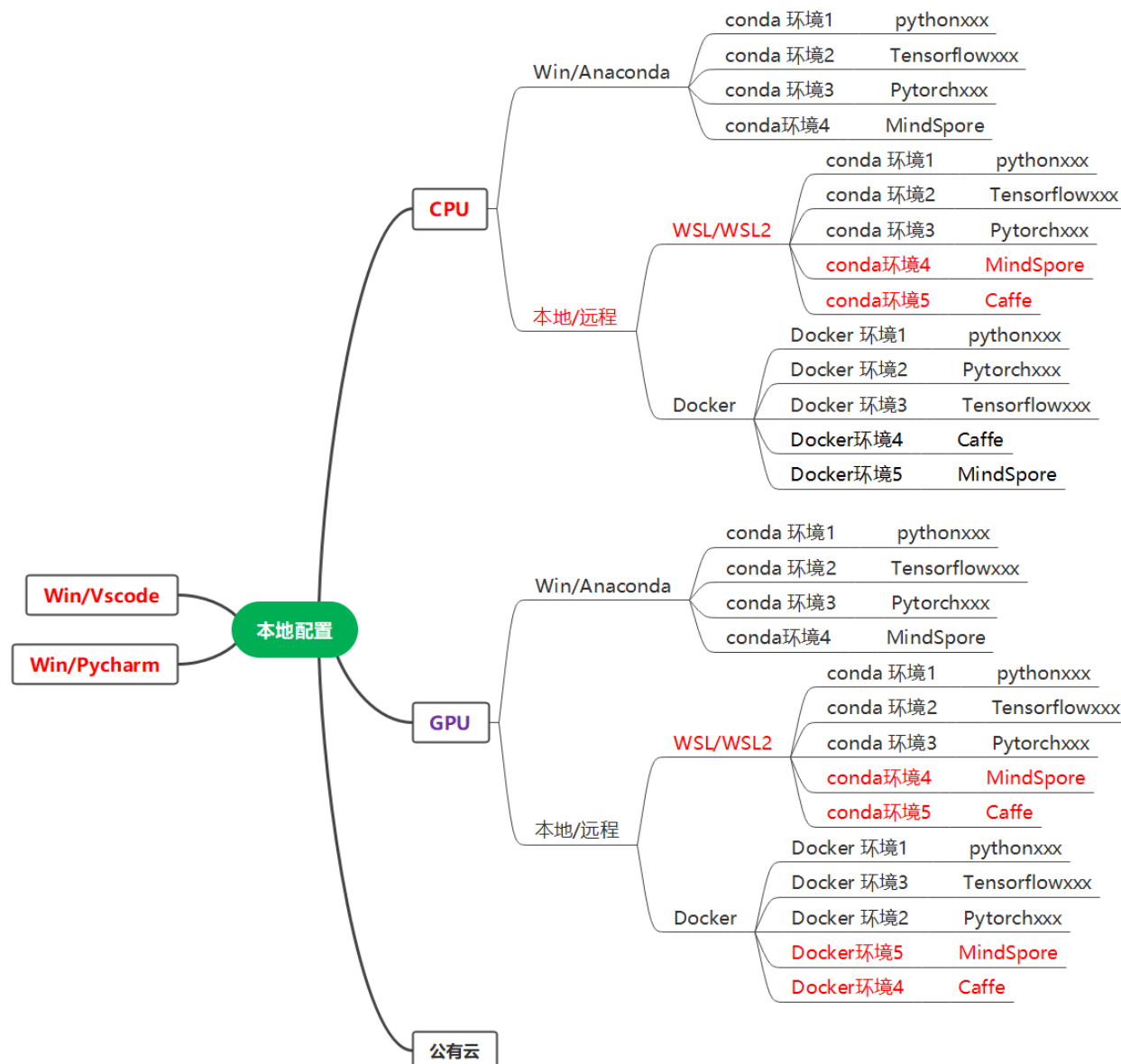
Eclipse

Eclipse 的本身只是一个框架平台，但是众多插件的支持使得 Eclipse 拥有其他功能相对固定的 IDE 软件很难具有的灵活性。许多软件开发商以 Eclipse 为框架开发自己的 IDE。



1.2 环境概述：各类实现方式

各类环境的实现方式





1.3 环境概述： 环境安装

- Anaconda 是一个广泛用于科学计算、数据分析、机器学习的 Python 和 R 发行版，由 Anaconda, Inc. 维护。它最大的特点是集成了大量常用的科学计算包，并提供了强大的包管理器 conda，极大地简化了 Python 环境管理和依赖安装的过程。
- Anaconda 的核心组件

组件名称	功能说明
Conda	包管理和环境管理工具，可以同时管理 Python 包和系统级依赖
Python/R	编程语言解释器
Jupyter Notebook	交互式笔记本，广泛用于数据分析和可视化
Spyder	面向科学计算的 IDE，类似 MATLAB
Anaconda Navigator	图形化界面，方便用户管理环境和启动应用
pip	Python 官方包管理器，和 conda 配合使用



1.3 环境概述： 环境安装

➤ Anaconda 的优势

- 一键安装大量科学包：避免手动安装依赖，节省时间。
- 环境隔离：使用 conda 可以为不同项目创建独立的 Python 环境，避免包冲突。
- 跨平台支持：支持 Windows、macOS 和 Linux。
- 图形化操作：Anaconda Navigator 提供图形界面，适合初学者。
- 社区活跃、文档丰富：有大量的教程和社区支持



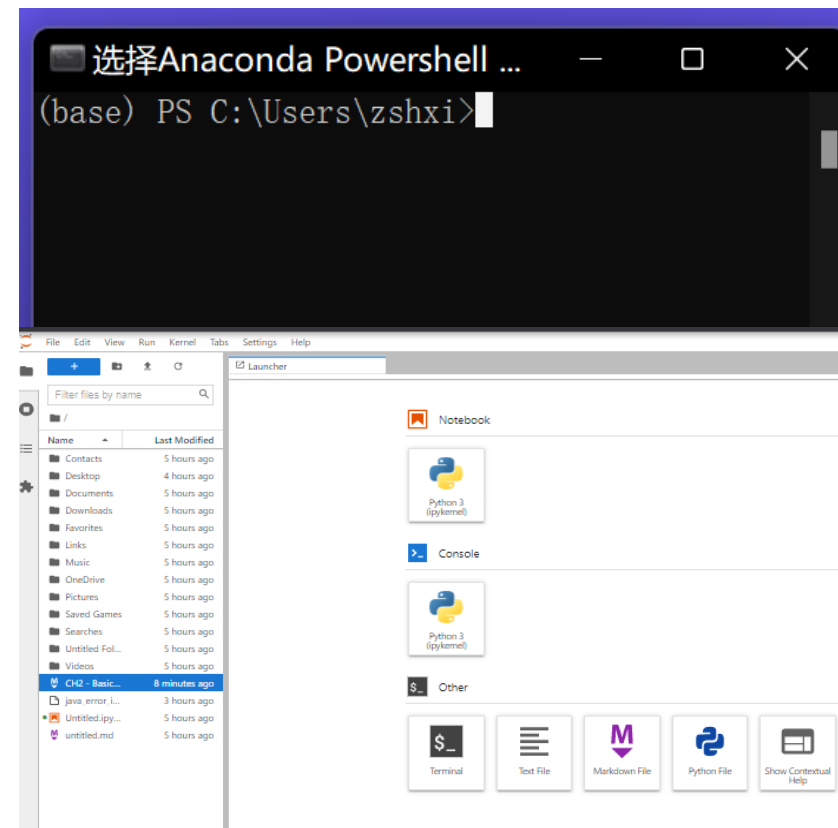
1.3 环境概述： 环境安装

- Windows下 Anaconda 安装(务必安装在D盘)
 - <https://www.anaconda.com/products/individual>
 - ✓ 下载 Anaconda个人版本
 - ✓ 点击安装
 - 其他可选安装：
 - ✓ 旧版本： [Index of /anaconda/archive/ | 清华开源软件镜像站](#)
 - ✓ Minconda： [Index of /anaconda/miniconda/ | 清华开源软件镜像站](#)



1.4 环境概述 环境安装: Anaconda 配置虚拟环境

- Win10, 右击 “开始”, 点击Anaconda power shell
Win11 点击 “开始”, 搜索Anaconda
运行: `conda info -e`
- `conda create --name ISP2025 python=3.7.5` #已经安装
- `conda activate ISP2025`
- # 以下 安装jupyterlab (选做)
- `conda install jupyterlab` #已经安装
- `jupyter lab` #运行
- 弹出jupyter lab 运行界面



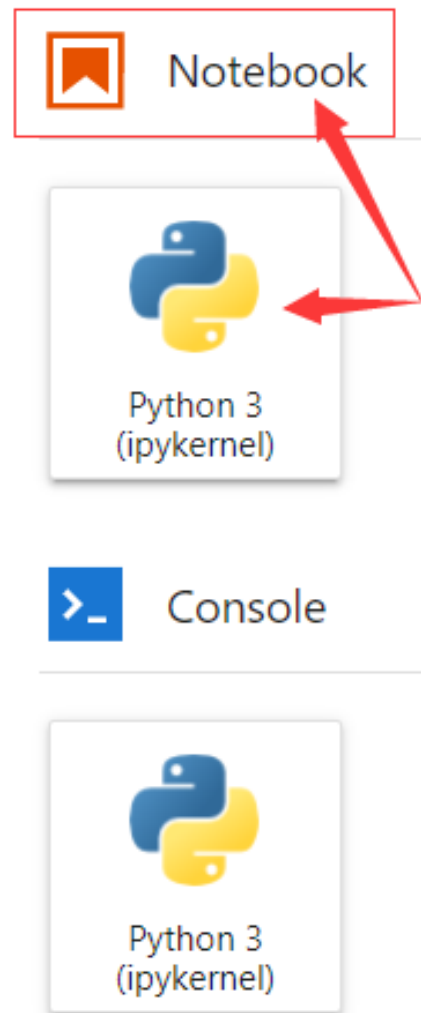


1.5 环境概述 环境安装: jupyter lab 使用

```
[I 2022-02-28 15:46:29.999 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2022-02-28 15:46:30.041 ServerApp]

To access the server, open this file in a browser:
    file:///C:/Users/zshxi/AppData/Roaming/jupyter/runtime/jpserver-8288-open.html
Or copy and paste one of these URLs:
    http://localhost:8888/lab?token=576d86690da2c03d245d5226d89464cfa49beabdd793f435
    or http://127.0.0.1:8888/lab?token=576d86690da2c03d245d5226d89464cfa49beabdd793f435
[W 2022-02-28 15:46:34.644 LabApp] Could not determine jupyterlab build status without nodejs
[I 2022-02-28 15:46:35.724 ServerApp] Kernel started: a051713b-b401-4644-9e5b-dclad6ff28db
[W 2022-02-28 15:46:36.817 ServerApp] Got events for closed stream <zmq.eventloop.zmqstream.ZMQStream object at 0x000002
0251E80888>
[I 2022-02-28 15:48:35.614 ServerApp] Saving file at /CH2 - Basic Data Type.md
[I 2022-02-28 16:05:43.820 ServerApp] Kernel started: 358bd251-61d3-491b-913b-5959a530680f
[I 2022-02-28 16:06:11.570 ServerApp] Starting buffering for 358bd251-61d3-491b-913b-5959a530680f:d68ca1ff-63f7-43b0-ac4
9-80ce8d9a1f84
```

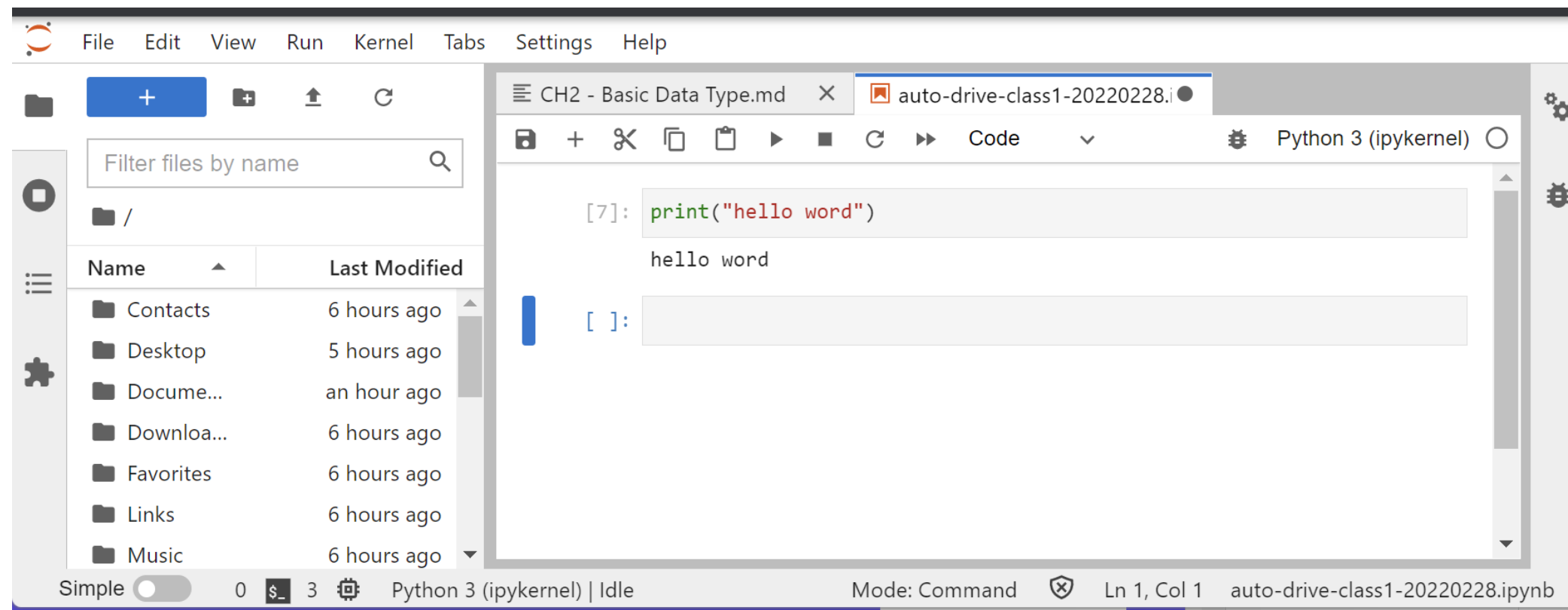
复制链接



在Notebook下点击python3，在弹出的Jupyternotebook文件里开始进行练习



1.5 环境概述 环境安装: jupyter lab 使用



关于jupyter lab 的进一步使用, 请参考: <https://docs.jupyter.org/en/latest/>



2

IDE介绍



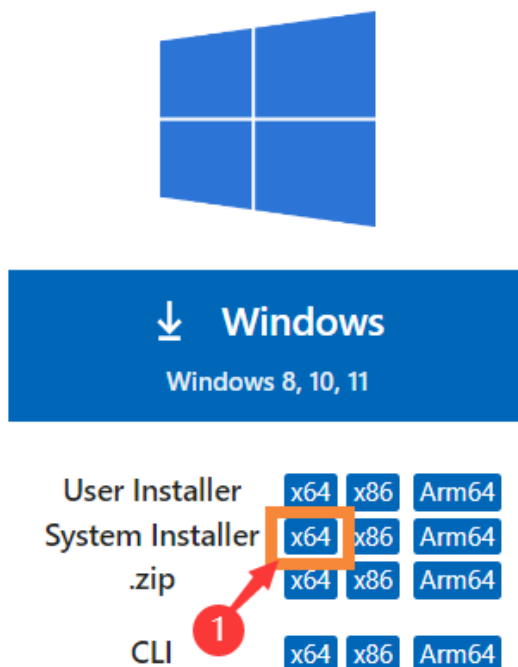
2. IDE介绍 Visual Studio Code (VS Code)

- VS Code 具有完备的代码开发、调试、管理功能，专门为提高编程速度而进行的一系列调整优化
 - 具有自动补全功能以及各种功能人性化的快捷键，可以方便的提升编程速度，改善编程体验
 - 支持Linux，在跨平台上迈出了一大步
 - 更为轻量化，更加方便实用，在任何平台上快速配置环境都更加方便
 - 独特的插件系统，使得VS Code的功能扩展有很多的可能性



2.2 IDE介绍 VS Code 安装

1. 官方下载页面--->: [VSCode官方下载页面链接](#), 选择自己系统对应的下载链接 (**Windows版本**)

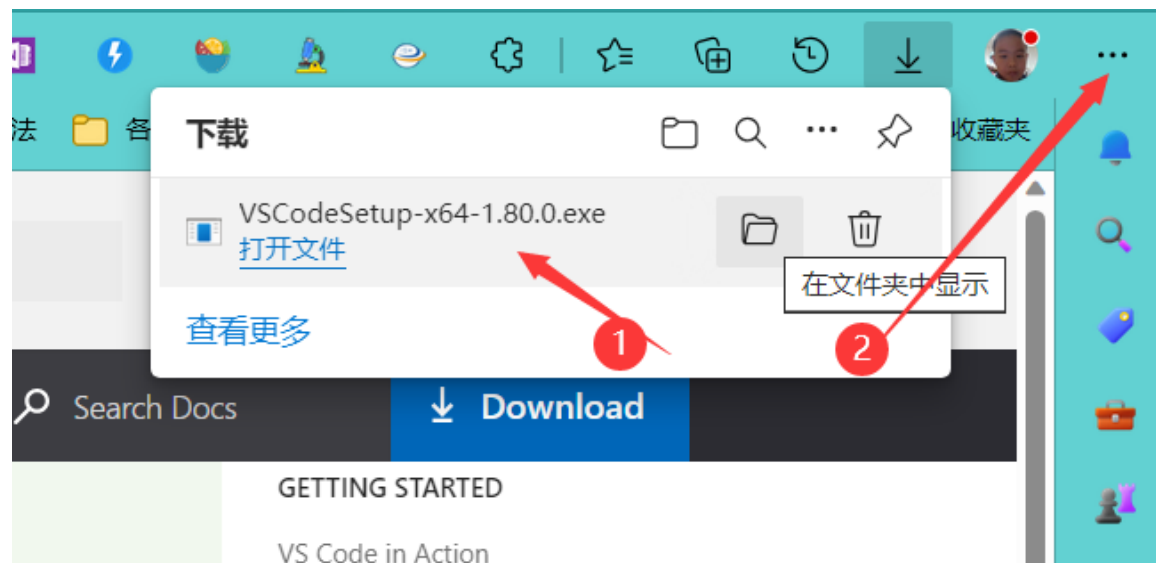


User Installer	默认安装在当前计算机帐户目录, 如果使用另一个帐号登陆计算机将无法使用别人安装的vscode。vscode默认提供的为 User Installer	默认安装
System Installer	安装在非用户目录,例如C盘根目录,任何帐户都可以使用	自定义安装路径,方便管理

2.2 IDE介绍 VS Code 安装



2. 下载 系统版本后，在浏览器右边找下载的文开始始安装（如果没有弹出，在点击2所示的“...”，在弹出界面找到并点击“下载”，即可弹出下载文件。按要求填写路径和开始菜单。

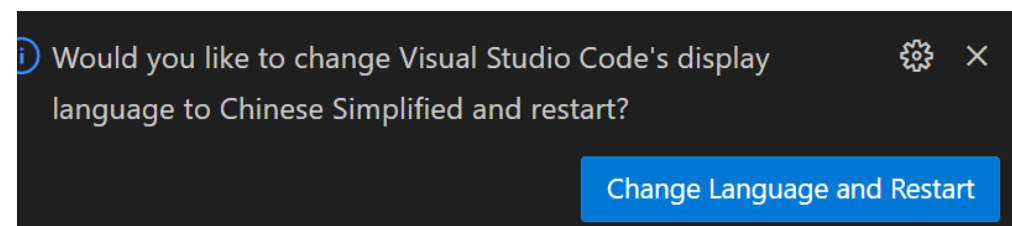


3. 选择附加任务（**务必勾选 PATH选项， 建议全部勾选**），开始安装

2.2 IDE介绍 VS Code 安装 语言包

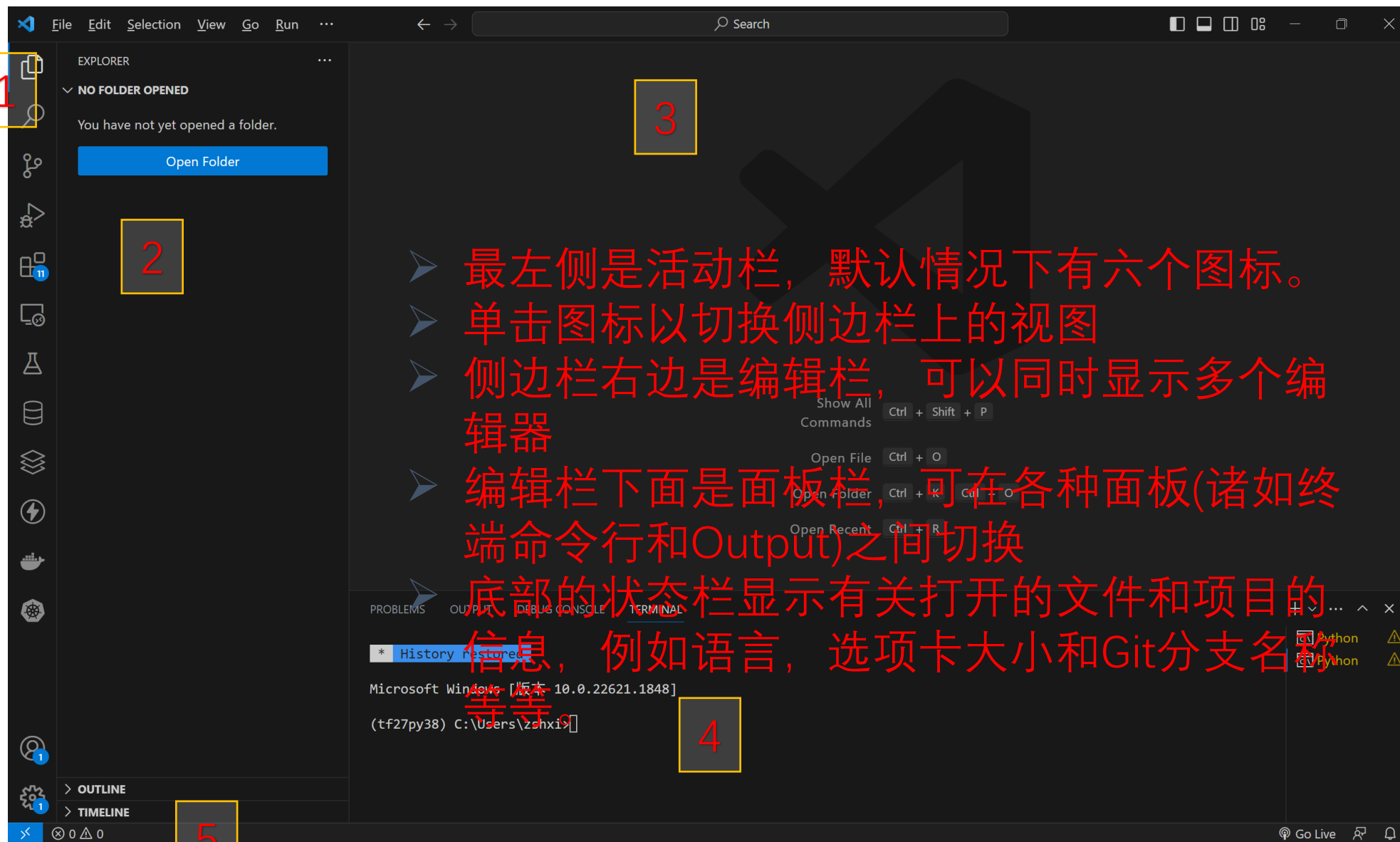


打开左侧插件栏（Extensions），在搜索框中输入chinese，安装后重启





2.3 IDE介绍 界面介绍



1. 活动栏
2. 侧边栏
3. 编辑栏
4. 面板栏
5. 状态栏

- 最左侧是活动栏，默认情况下有六个图标。
- 单击图标以切换侧边栏上的视图
- 侧边栏右边是编辑栏，可以同时显示多个编辑器
- 编辑栏下面是面板栏，可在各种面板(诸如终端命令行和Output)之间切换
- 底部的状态栏显示有关打开的文件和项目信息，例如语言，选项卡大小和Git分支名称等等。



2.3 IDE介绍 侧边栏

“Ctrl+B”可关闭或打开侧边栏
通过如下步骤切换左右显示位置。

如右图通过菜单命令打开“设置”





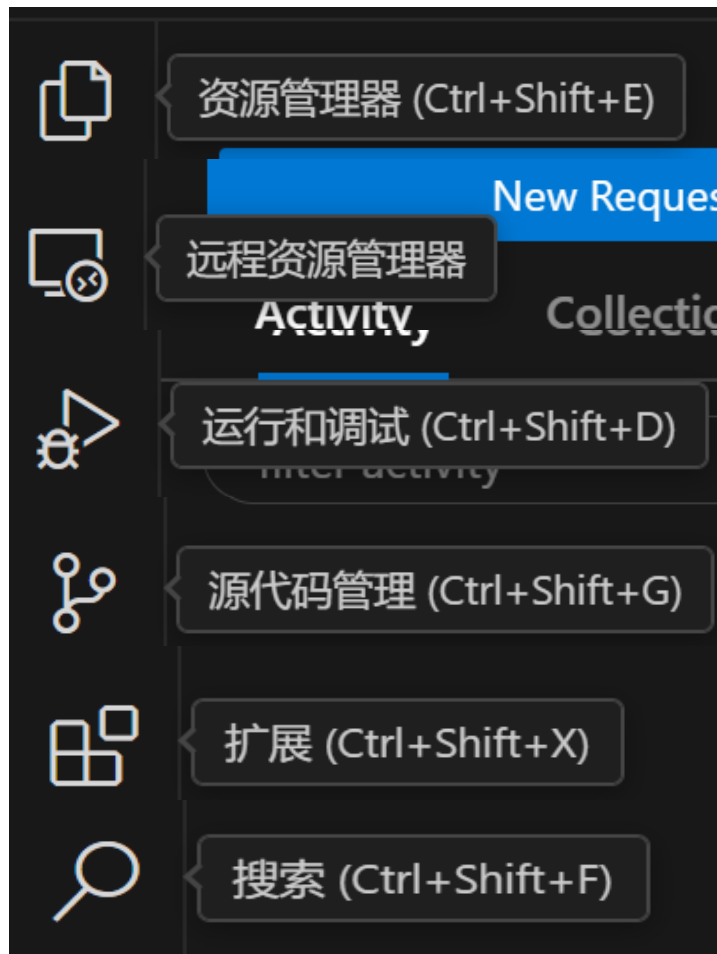
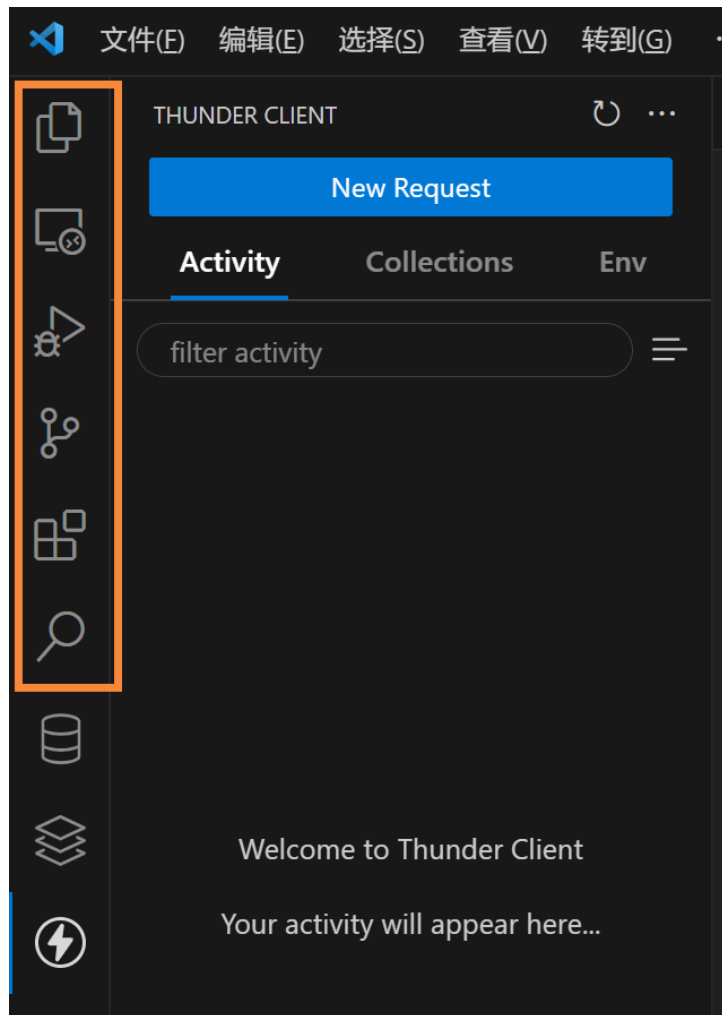
2.3 IDE介绍 资源管理器





2.3 IDE介绍 活动栏

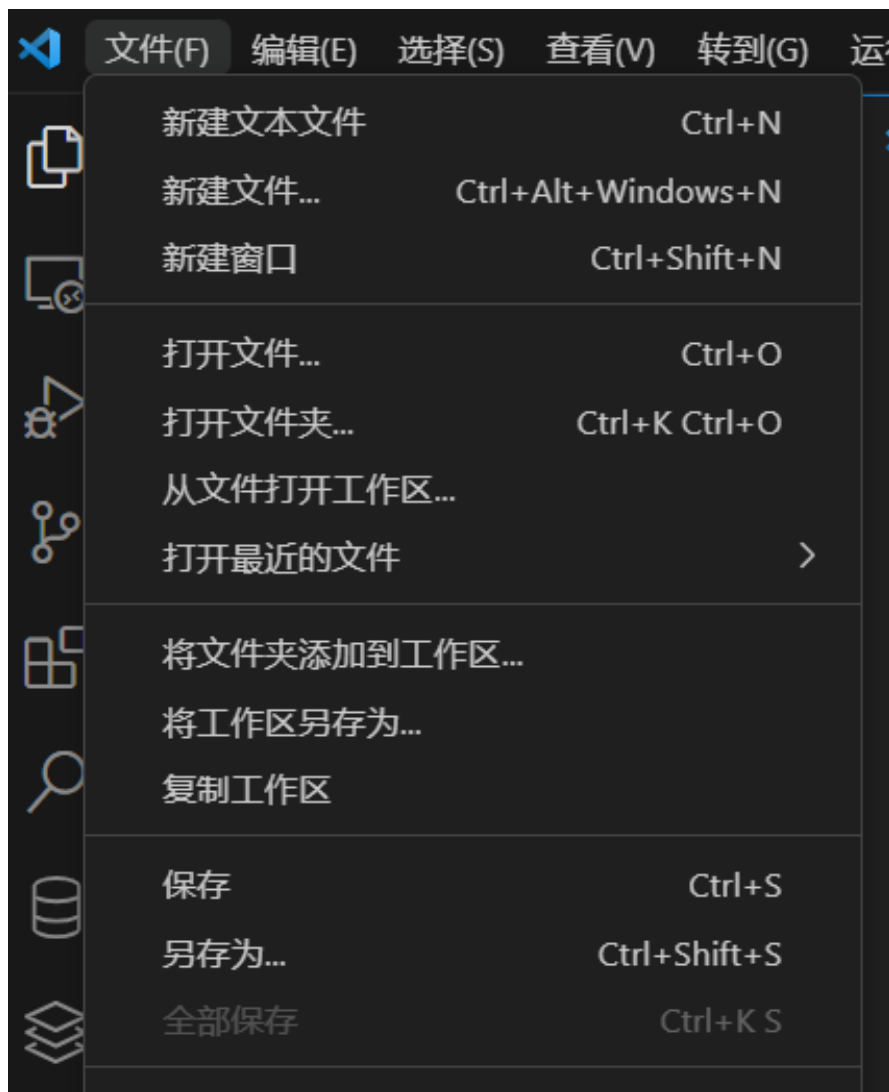
活动栏默认有六个图标。



- 添加的扩展功能可能会追加新图标在活动栏上。
- 上下拖动图标可切换位置
- 右键单击活动栏, 可在关联菜单上显示或隐藏图标

2.3 IDE介绍 工作区

- 在VSCode中工作区代表了一系列目录的集合。
- 执行菜单命令【文件】【将文件夹添加到工作区】
- 可以在工作区中打开多个不同目录的。多个目录并列显示在侧边栏
- 执行菜单命令【文件】【将工作区另存为】



2.3 IDE介绍 面板



面板通常显示在右下部分，默认有问题，输出，调试控制台，终端四种。根据安装的扩展，有可能会追加新的面板

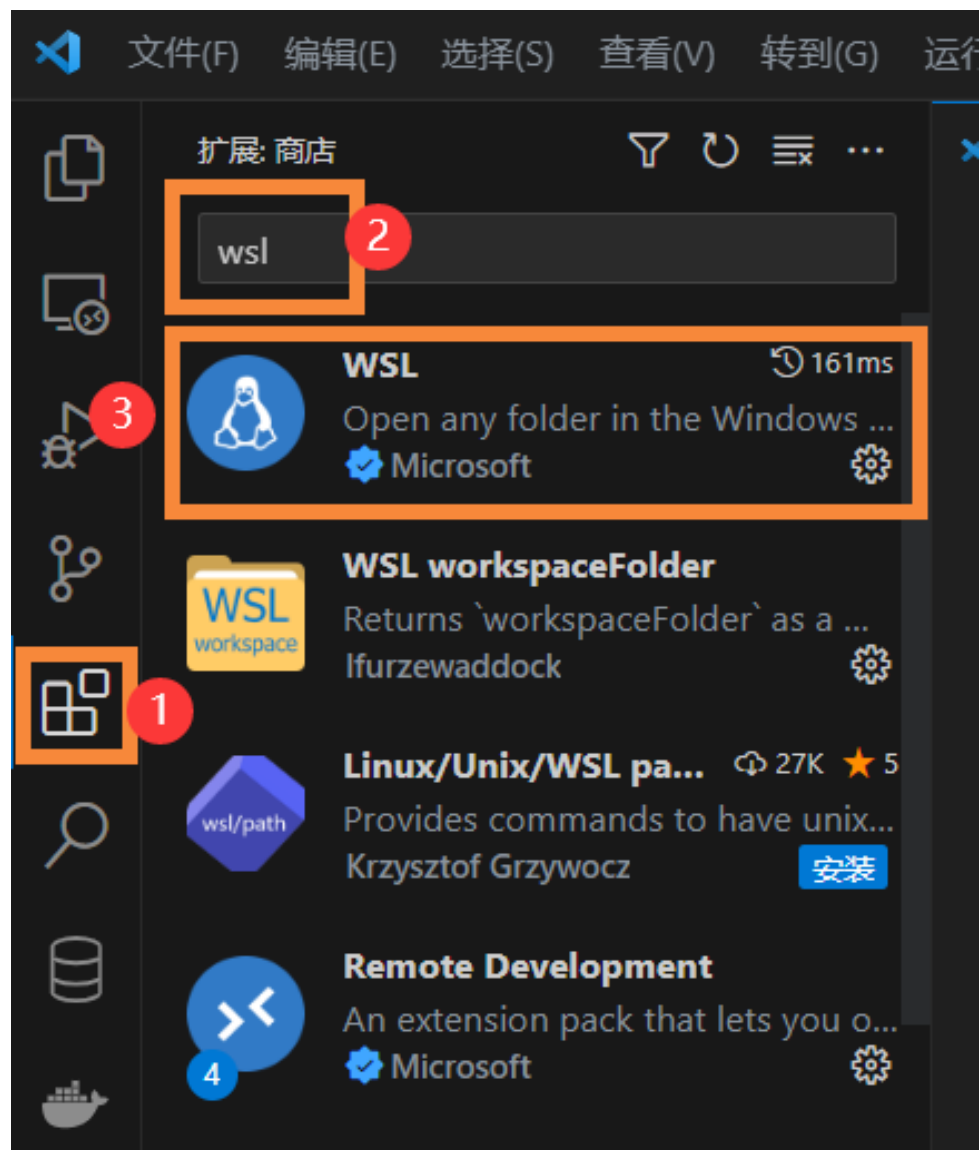
终端面板可以在vsc内部开启OS终端

运行shell等命令

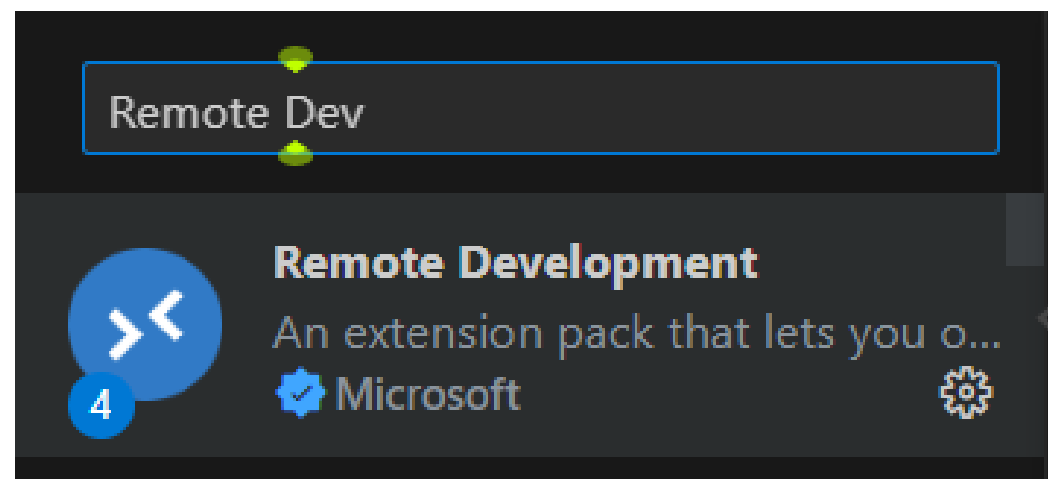
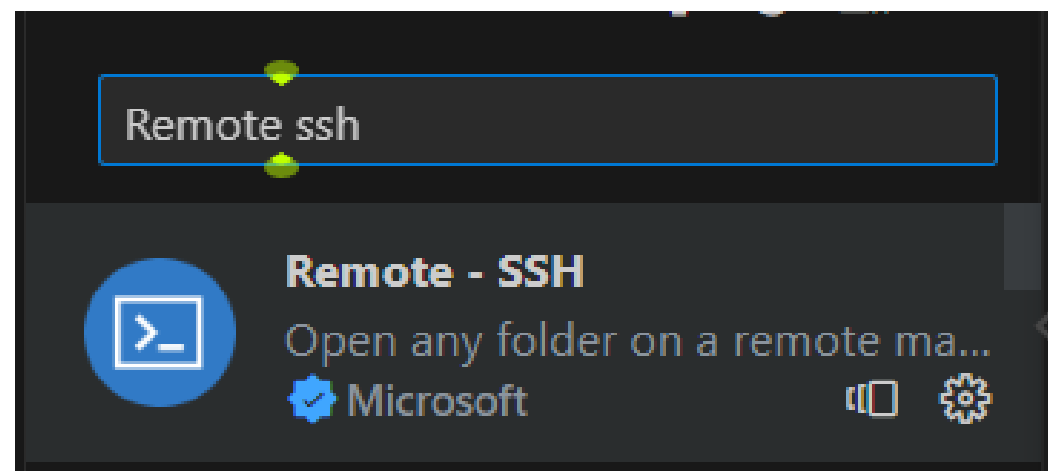
在终端里可以直接运行os终端命令

默认工作目录为vsc当前工作区的目录

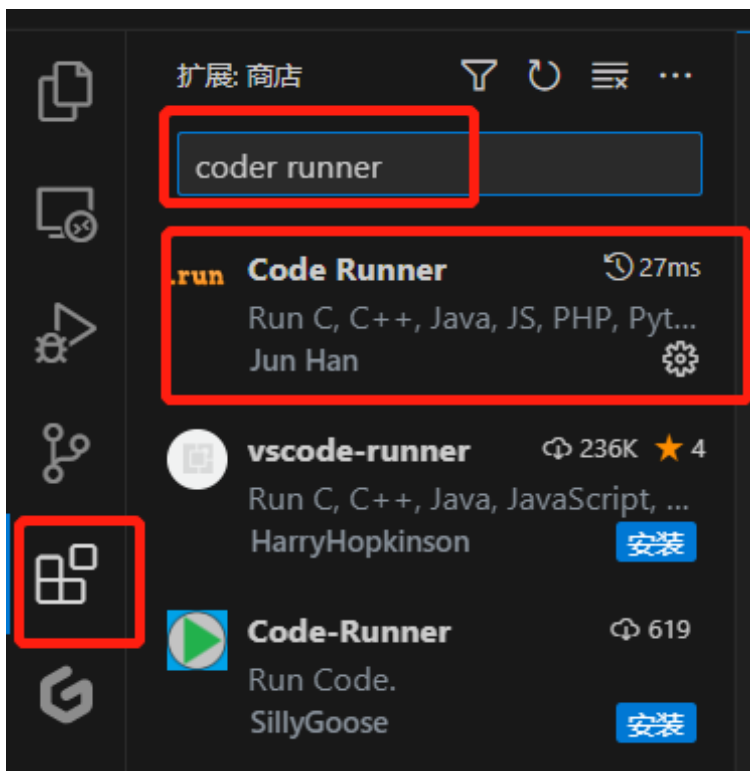
2.4 IDE扩展：安装WSL扩展



如左图，安装WSL扩展工具，如下两图，安装剩下的扩展工具



2.4 IDE扩展：Code Runner安装





2.4 IDE扩展 SSH远程连接

The image shows a multi-panel view of the Visual Studio Code interface during the setup of the Remote SSH extension. The left sidebar displays the 'Remote - SSH' extension by Microsoft, which is highlighted with a red box. Below it, the 'Remote Explorer' view is also highlighted with a red box. The main editor area shows the 'Remote (Tunnel/SSH)' view, where a list of SSH hosts is displayed, including '192.168.1.2', '172.24.44.116', '10.26.35.235', '192.168.1.100', and 'davinic-mini'. A red box highlights the 'SSH' section in the sidebar. On the right, a terminal window shows the command 'ssh hello@microsoft.com -A' and a message indicating the SSH host '192.168.1.2' is being initialized. Below the terminal, a dialog box prompts the user to 'Select the platform of the remote host "192.168.1.2"', with 'Linux' selected and highlighted by a red box.

文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R)

扩展: 商店

Remote ssh

Remote - SSH 105ms
Open any folder on a remote ...
Microsoft

Remote - SSH: Editing Conf...
Edit SSH configuration files
Microsoft

Remote Explorer
View remote machines for SS...

远程资源管理器

远程(隧道/SSH)

登录以查看注册的隧道

SSH

192.168.1.2
172.24.44.116
10.26.35.235
192.168.1.100
davinic-mini

username@目标IP 输入 SSH 连接命令

E.g. ssh hello@microsoft.com -A

按 "Enter" 以确认或按 "Esc" 以取消

设置 SSH 主机 192.168.1.2: (details) 正在初始化 VS Code 服务器

[10:15:49.862] Remote-SSH version: remote-ssh@0.103.2023071415
[10:15:49.862] win32 x64
[10:15:49.863] SSH Resolver called for host: 192.168.1.2
[10:15:49.863] Setting up SSH remote "192.168.1.2"
[10:15:49.865] Using commit id "660393deaaa6d1996740ff4880f1bad43768c814" and quality "stable" for server
[10:15:49.867] Install and start server if needed

Select the platform of the remote host "192.168.1.2"

Linux
Windows
macOS

2.4 快捷键



快捷键	说明
Ctrl+Shift+P, F1	显示命令选项板
Ctrl+P	快速打开, 转到文件...
Ctrl+Shift+N	新建窗口/实例
Ctrl+Shift+W	关闭窗口/实例
Ctrl+	用户设置
Ctrl+K Ctrl+S	键盘快捷键

[更多的快捷键](#)



3

Python虚拟环境

3 Python虚拟环境 安装Anaconda (输入时巧用tab 补全)



1. win11 单击“开始”，弹出的搜索框输入“Ubuntu”，点击“Ubuntu20.04 LTS”
打开WSL2 （如果没有，则需要去Microsoft store 里查找并安装）
2. 在Ubuntu20.04 发行版安装SSH工具和wget 工具。

```
sudo apt-get install openssh-server
```

```
apt-get install -y wget
```

3. 下载ananoconda，并修改下载文件的权限为可执行

```
cd /home/xxx(此处为用户目录)
```

```
wget https://repo.anaconda.com/archive/Anaconda3-2023.03-1-Linux-x86\_64.sh
```

```
chmod +x Anaconda3-2023.03-1-Linux-x86_64.sh
```



3 Python虚拟环境 安装Anaconda

4. 安装: `./Anaconda3-2023.03-1-Linux-x86_64.sh`

5. 弹出如下界面后, 点击“Enter”

```
Welcome to Anaconda3 2023.03-1

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>> |
```

页面左下方出现“More--”时候, 一直按enter, 直至问我们是否接受授权条款的时候, 输入“yes”

```
Do you accept the license terms? [yes|no]
[no] >>> |
```

6. 接受安装位置, 按“ENTER”

```
Anaconda3 will now be installed into this location:
/root/anaconda3

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/root/anaconda3] >>> |
```

3 Python虚拟环境 安装Anaconda



7. 输入“yes” 接受对Ananconda 的初始化

```
done
installation finished.
Do you wish the installer to initialize Anaconda3
by running conda init? [yes|no]
[no] >>> |
```

8. 关掉当前的Ubunt18.04 LTS 发行版本，重新进入后，输入“conda -V”,如果出现版本号，则安装完成。

```
(base) root@LAPTOP-83HOTT8M:~# conda -V
conda 23.3.1
(base) root@LAPTOP-83HOTT8M:~# |
```

3.2 Python虚拟环境 Anaconda命令



获取版本号	conda -V
	conda --version
获取帮助	conda -h
	conda env --help
获取环境相关命令的帮助	conda env -h
所有 --单词 都可以用 -单词 首字母来代替	比如 -version 可以用 -V来代 替，只不过有的是大写，有的 可能是小写



3.2 Python虚拟环境 Anaconda命令

创建环境	<code>conda create -n environment_name</code>
创建指定python版本下包含某些包的环境	<code>conda create -n environment_name python=3.7 numpy scipy</code>
进入环境	<code>conda activate environment_name</code>
退出环境	<code>conda deactivate</code>
删除环境	<code>conda remove -n yourname --all</code>
列出环境	<code>conda env list / conda info -e</code>
复制环境	<code>conda create --name new_env_name --clone old_env_name</code>
指定目录下生成环境yml文件	<code>conda env export > 目录/environment.yml</code>
从yml文件创建环境	<code>conda env create -n env_name -f environment.yml</code>



3.2 Python虚拟环境 Anaconda命令

安装包	conda instal package_name
查看当前环境包列表	conda list
查看指定环境包列表	conda list -n environment_name
查看conda源中包的信息	conda search package_name
更新包	conda update package_name
删除包	conda remove package_name
清理无用的安装包	conda clean -p
清理tar包	conda clean -t
清理所有安装包及cache	conda clean -y --all
更新anaconda	conda update annaconda



3.3 Python虚拟环境 更换conda源

windows: 命令行中添加以下命令:

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/free/  
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/  
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/  
conda config --set show_channel_urls yes
```

```
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/pkgsg/main/  
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/pkgsg/free/  
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/cloud/conda-forge/  
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/cloud/msys2/  
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/cloud/bioconda/  
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/cloud/menpo/  
conda config --set show_channel_urls yes
```

Linux:

将以上两种配置之一写在~/.condarc 中: vim ~/.condarc



3.4 Python虚拟环境 Anaconda 安装实践

进入Ubuntu20.04 子系统后:

1. `conda env list` //查看当前虚拟环境
2. `conda create -n ISP2025 python=3.7` //创建名为ISP_Practice 的虚拟环境, 并安装python3.7
3. 待弹出安装确认后, 输入“y”
4. `conda env list`
5. `conda activate ISP2025` //激活虚拟环境
6. 输入“Python”, 如下图所示:

```
(base) root@LAPTOP-83H0TT8M:/home/auto# conda activate IS_Practice
(IS_Practice) root@LAPTOP-83H0TT8M:/home/auto# python
Python 3.7.16 (default, Jan 17 2023, 22:20:44)
[GCC 11.2.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

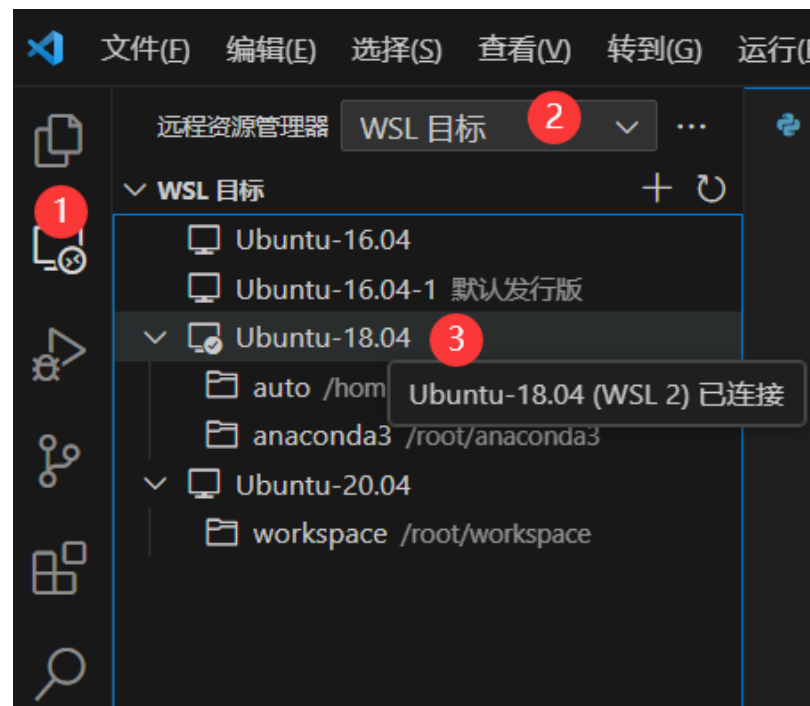


4 Python 实例

4 Python 实例



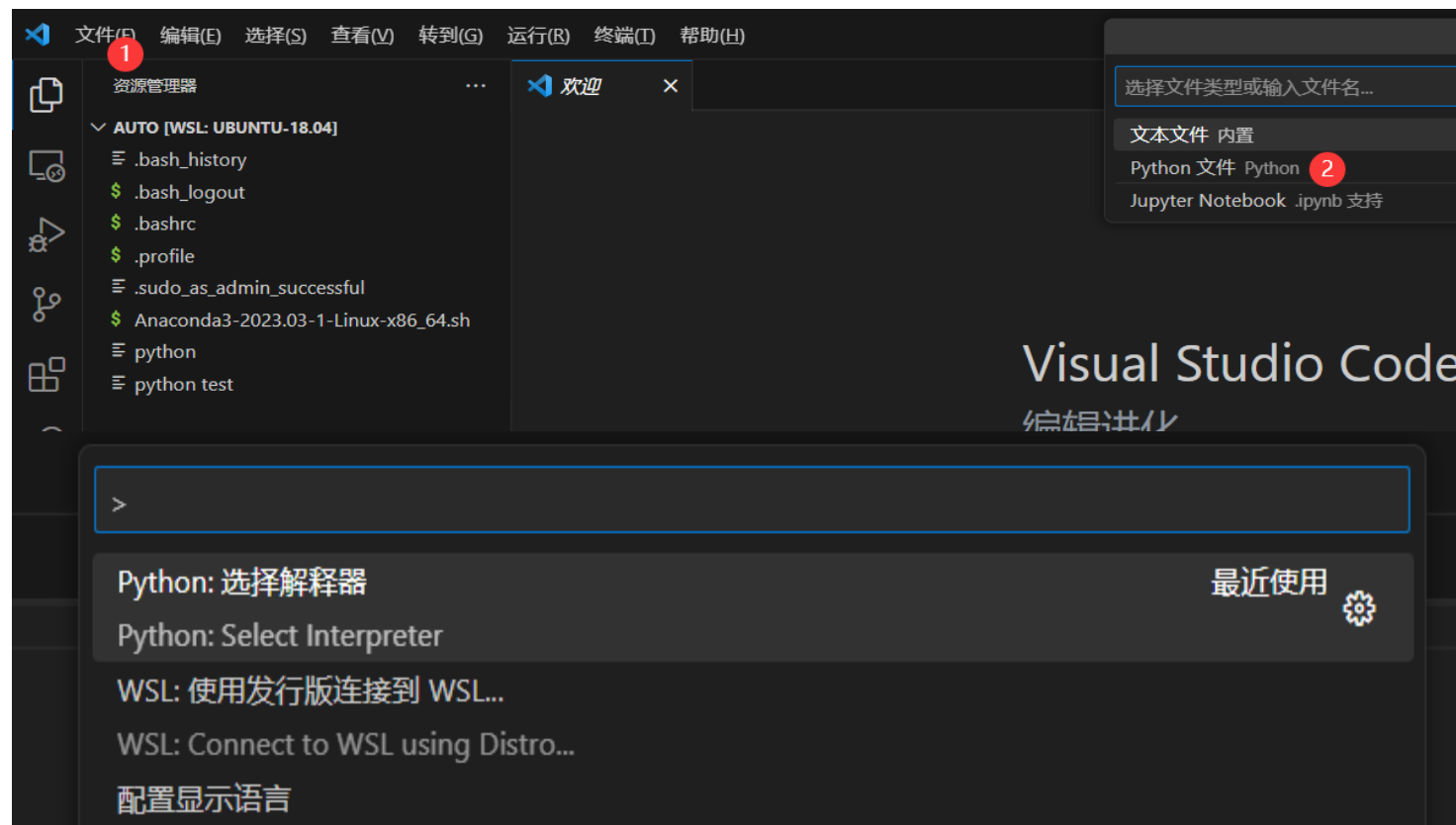
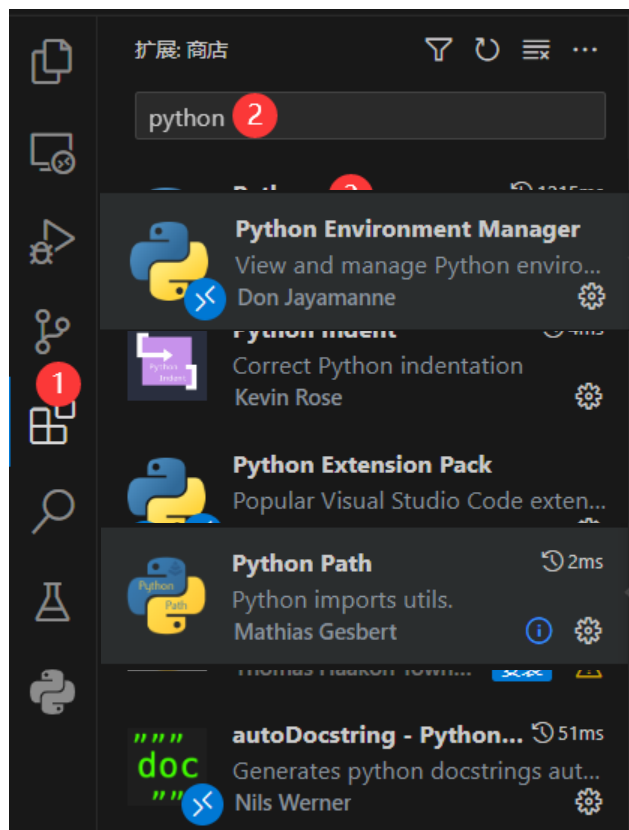
1. VS Code 打开WSL2 分发,
- 在子系统运行python
2. 如下图顺次连接Ubuntu20.04子系统。



4 Python 实例 实践



安装4个python相关插件、连接WSL2，新建python文件、选择Python解释器





4 Python 实例 实践 绘制一个五角星

- 科赫曲线的基本概念和绘制方法如下：
- 正整数 n 代表科赫曲线的阶数，表示生成科赫曲线过程的操作次数。
 - 科赫曲线初始化阶数为0，表示一个长度为 L 的直线。
 - 对于直线 L ，将其等分为三段，中间一段用边长为 $L/3$ 的等边三角形的两个边替代，得到1阶科赫曲线，它包含四条线段。
 - 进一步对每条线段重复同样的操作后得到2阶科赫曲线。继续重复同样的操作 n 次可以得到 n 阶科赫曲线。

0阶科赫曲线



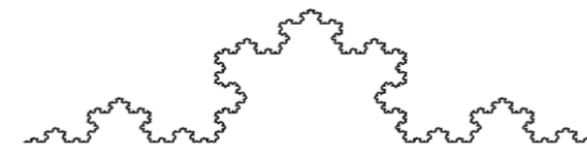
1阶科赫曲线



2阶科赫曲线



5阶科赫曲线

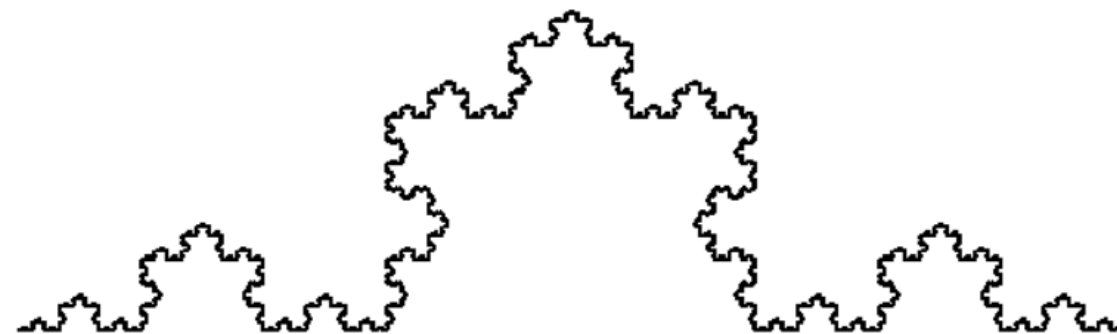


4 Python 实例 实践 绘制一个五角星



```
import turtle
def koch(size,n):
    if n==0:
        turtle.fd(size)
    else:
        for angle in[0,60,-120,60]:
            turtle.left(angle)
            koch(size/3,n-1)
def main():
    turtle.setup(800,400)
    turtle.speed(0)#控制绘制速度
    turtle.penup()
    turtle.goto(-300,-50)
    turtle.pendown()
    turtle.pensize(2)
    koch(600,6)#0阶科赫曲线长度, 阶数
    turtle.hideturtle()
```

main()





05 Python 库



5.1 Python 库: os 库

os库提供通用的、基本的操作系统交互功能

os库是python标准库, 包含几百个函数

常用路径操作、进程管理、环境参数等几类

路径操作: os.path子库, 处理文件路径及信息

进程管理: 启动系统中其他程序

环境参数: 获得系统软件硬件信息等环境参数

os.abspath(path)	# 返回path在当前系统中的绝对路径
os.normpath(path)	# 归一化path的表示形式, 统一用\\分隔路径
os.relpath(path)	# 返回当前程序与文件之间的相对路径 (relative path)
os.dirname(path)	# 返回path中的目录名称
os.basename(path)	# 返回path中最后的文件名称
os.join(path, *paths)	# 组合path与paths, 返回一个路径字符串
os.exists(path)	# 判断path对应文件或目录是否存在, 返回True或False
os.isfile(path)	# 判断path所对应是否为已存在的文件, 返回True或False
os.isdir(path)	# 判断path所对应是否为已存在的目录, 返回True或False
os.getatime(path)	# 返回path对应文件或目录上一次的访问时间
os.getmtime(path)	# 返回path对应文件或目录最近一次的修改时间
os.getctime(path)	# 返回path对应文件或目录的创建时间
os.getsize(path)	# 返回path对应文件的大小, 以字节为单位
os.chdir(path)	# 修改当前程序操作的路径
os.getcwd()	# 返回程序的当前路径
os.getlogin()	# 获得当前系统登录用户名称
os.cpu_count()	# 获得当前系统的CPU数量
os.urandom(n)	# 获得n个字节长度的随机字符串, 通常用于加解密运算



5.2 Python 库 Argparse 介绍

➤ argparse是python用于解析命令行参数和选项的标准模块,基本用法如下:

- 1: import argparse
- 2: parser = argparse.ArgumentParser()
- 3: parser.add_argument()
- 4: parser.parse_args()

- 首先导入该模块;
- 其次创建一个解析对象;
- 再次向该对象中添加你要关注的命令行参数和选项, 每一个add_argument方法对应一个你要关注的参数或选项;
- 最后调用parse_args()方法进行解析;
- 解析成功之后即可使用。
- 可以 传入一个参数、传入多个参数、操作args字典、传入多个参数、改变数据类型、位置参数、可选参数、默认值、必需参数

➤ 创建demo.py文件, 输入如下内容:

```
import argparse
parser = argparse.ArgumentParser(description='命令行中传入一个数字')
#type是要传入的参数的数据类型 help是该参数的提示信息
parser.add_argument('integers', type=str, help='传入的数字')
args = parser.parse_args()
#获得传入的参数
print(args)
```

```
>>python demo.py -h 5
Namespace(integers='5')
```

```
>>python demo.py -h
usage: demo.py [-h] integers
命令行中传入一个数字
positional arguments:
  integers  传入的数字
optional arguments:
  -h, --help  show this help message and exit
```



5.3 Python 库 Python官方文档

← → ↺ 🏠 <https://docs.python.org/zh-cn/3.10/index.html>

📁 谢燕相关 📁 华为智能基座相关 📁 FPGA 学习 📁 各类嵌入式硬件及... 📁 预测算法 📁 Linux 应用 📁 Tensorflow 📁 Pytorch 📁 各类算法

Python » Simplified Chinese » 3.10.4 » 3.10.4 Documentation »

下载

下载这些文档

各版本文档

- Python 3.11 (in development)
- Python 3.10 (stable)
- Python 3.9 (stable)
- Python 3.8 (security-fixes)
- Python 3.7 (security-fixes)
- Python 3.6 (EOL)
- Python 3.5 (EOL)
- Python 2.7 (EOL)
- 所有版本

其他资源

- PEP 索引
- 初学者指南 (维基)
- 推荐书单
- 音视频小讲座
- Python 开发者指南

Python 3.10.4 文档

欢迎! 这里是 Python 3.10.4 的官方文档。

按章节浏览文档:

Python 3.10 有什么新变化?
或显示自 2.0 以来的全部新变化

教程
从这里看起

标准库参考
放在枕边作为参考

语言参考
讲解基础内容和基本语法

Python安装和使用
各种操作系统的介绍都有

Python 常用指引
深入了解特定主题

索引和表格:

全局模块索引
快速查看所有的模块

总目录
所有的函数, 类, 术语

术语对照表
解释最重要的术语

安装 Python 模块
从官方的 PyPI 或者其他来源安装模块

分发 Python 模块
发布模块, 供其他人安装

扩展和嵌入
给 C/C++ 程序员的教程

Python/C API 接口
给 C/C++ 程序员的参考手册

常见问题
经常被问到的问题 (答案也有!)

搜索页面
搜索文档

完整的内容表
列出所有的章节和部分

<https://docs.python.org/zh-cn/3.10/index.html>



5.4 Python 库 Numpy

- NumPy是使用Python进行科学计算的基础软件包。它包括:
 - 功能强大的N维数组对象。
 - 精密广播功能函数。
 - 集成 C/C+和Fortran 代码的工具。
 - 强大的线性代数、傅立叶变换和随机数功能。
- NumPy 最重要的一个特点是其 N 维数组对象 ndarray，它是一系列同类型数据的集合，以 0 下标为开始进行集合中元素的索引。ndarray 对象是用于存放同类型元素的多维数组。ndarray 中的每个元素在内存中都有相同存储大小的区域。
- ndarray对象的内容可以通过索引或切片来访问和修改，与 Python 中 list 的切片操作一样。ndarray 数组可以基于 0 - n 的下标进行索引，切片对象可以通过内置的 slice 函数，并设置 start, stop 及 step 参数进行，从原数组中切割出一个新数组。

```
>>> import numpy as np
>>> a = np.arange(15).reshape(3, 5)
>>> a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
>>> a.shape
(3, 5)
>>> a.ndim
2
>>> a.dtype.name
'int64'
>>> a.itemsize
8
>>> a.size
15
>>> type(a)
<type 'numpy.ndarray'>
>>> b = np.array([6, 7, 8])
>>> b
array([6, 7, 8])
>>> type(b)
<type 'numpy.ndarray'>
```



5.4 Python 库 Numpy

➤ NumPy的主要对象是同构多维数组。它是一个元素表（通常是数字），所有类型都相同，由非负整数元组索引。在NumPy维度中称为 轴。

➤ 3D空间中的点的坐标[1, 2, 1]具有一个轴。该轴有3个元素，所以我们说它的长度为3.在下图所示的例子中，数组有2个轴。第一轴的长度为2，第二轴的长度为3。

```
[[ 1., 0., 0.],  
 [ 0., 1., 2.]]
```

➤ NumPy的数组类调用ndarray。ndarray对象更重要的属性是：

- ndarray.ndim - 数组的轴（维度）的个数。在Python世界中，维度的数量被称为rank。
- ndarray.shape - 数组的维度。这是一个整数的元组，表示每个维度中数组的大小。对于有 n 行和 m 列的矩阵，shape 将是 (n,m)。因此，shape 元组的长度就是rank或维度的个数 ndim。
- ndarray.size - 数组元素的总数。这等于 shape 的元素的乘积。
- ndarray.dtype - 一个描述数组中元素类型的对象。可以使用标准的Python类型创建或指定dtype。另外NumPy提供它自己的类型。例如numpy.int32、numpy.int16和numpy.float64。
- ndarray.itemsize - 数组中每个元素的字节大小。例，元素为 float64 类型的数组的 itemsize 为8（=64/8），而 complex32 类型的数组的 itemsize 为4（=32/8）。它等于 ndarray.dtype.itemsize。
- ndarray.data - 该缓冲区包含数组的实际元素。通常不需要使用此属性，因为使用索引访问数组中的元素



5.4 Python 库 Numpy

➤ 数组创建

```
>>> import numpy as np
>>> a = np.array([2,3,4])
>>> a
array([2, 3, 4])
>>> a.dtype
dtype('int64')
>>> b = np.array([1.2, 3.5, 5.1])
>>> b.dtype
dtype('float64')
>>> np.zeros( (3,4) )
array([[ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.]])
>>> np.ones( (2,3,4), dtype=np.int16 )
# dtype can also be specified
array([[[ 1, 1, 1, 1],
       [ 1, 1, 1, 1],
       [ 1, 1, 1, 1],
       [ 1, 1, 1, 1],
       [ 1, 1, 1, 1],
       [ 1, 1, 1, 1]]], dtype=int16)
```

➤ 基本操作

```
>>> a = np.array( [20,30,40,50] )
>>> b = np.arange( 4 )
>>> b
array([0, 1, 2, 3])
>>> c = a-b
>>> c
array([20, 29, 38, 47])
>>> b**2
array([0, 1, 4, 9])
>>> 10*np.sin(a)
array([ 9.12945251, -9.88031624,
       7.4511316 , -2.62374854])
>>> a<35
array([ True,  True, False, False])
```

➤ 索引、切片和迭代

```
>>> a = np.arange(10)**3
>>> a
array([ 0,  1,  8, 27, 64, 125, 216, 343, 512,
       729])
>>> a[2]
8
>>> a[2:5]
array([ 8, 27, 64])
>>> a[:6:2] = -1000 # equivalent to
a[0:6:2] = -1000; from start to position 6,
exclusive, set every 2nd element to -1000
>>> a
array([-1000,   1, -1000,  27, -1000,  125,
       216,  343,  512,  729])
>>> a[::-1] # reversed a
array([ 729,  512,  343,  216,  125, -1000,
       27, -1000,   1, -1000])
```




5.4 Python 库 Numpy

- 多维的数组每个轴可以有一个索引。这些索引以逗号分隔的元组给出

```
>>> def f(x,y):
...     return 10*x+y
...
>>> b = np.fromfunction(f,(5,4),dtype=int)
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])

>>> b[2,3]
23

>>> b[0:5, 1] # each row in the second column of b
array([ 1, 11, 21, 31, 41])

>>> b[:,1] # equivalent to the previous example
array([ 1, 11, 21, 31, 41])

>>> b[1:3, :] # each column in the second and third row of b
array([[10, 11, 12, 13],
       [20, 21, 22, 23]])
```

- 当提供的索引少于轴的数量时，缺失的索引被认为是完整的切片：

```
>>> b[-1] # the last row. Equivalent to b[-1,:]
array([40, 41, 42, 43])
b[i] 方括号中的表达式 i 被视为后面紧跟着 : 的多个实例，用于表示剩余轴。
```

- NumPy也允许你使用三个点写为 `b[i,...]`。三个点 (...) 表示产生完整索引元组所需的冒号。例如，如果 `x` 是rank为5的数组（即，它具有5个轴），则：

- `x[1,2,...]` 相当于 `x[1,2,:,:,:]`,
- `x[... ,3]` 等效于 `x[:, :, :, :, 3]`
- `x[4,...,5,:]` 等效于 `x[4, :, :, 5, :]`。

- 对多维数组进行 迭代 (Iterating) 是相对于第一个轴完成的：

```
>>> for row in b:
...     print(row)
...
[0 1 2 3]
[10 11 12 13]
[20 21 22 23]
[30 31 32 33]
[40 41 42 43]
```

6 练习



- 1. 完成WSL2 Ubuntu20.04 下Anaconda 安装。并在WSL2 的Ubuntu20.04 中创建 python虚拟环境（自拟名字，建议带必要的版本号）
- 2. Vscode 添加WSL组件；
- 3. Vscode 打开WSL2 远程环境，进行python编程（内容：**绘制一个五角星**），对创建的py文件选择 wsl2 python虚拟环境里的解释器。
- 4. 运行python 得到输出结果。
- 5. 查找资料、熟悉numpy、pandas用法。